Phase 2: The Integer Engine and The Noise Crisis

Fully Homomorphic Encryption over the Integers

Paper Presentation

April 13, 2026

Outline: Phase 2

1. The Symmetric & Public-Key SHE Schemes
2. Rigorous Proof of Correctness
3. The Multiplicative Bottleneck
4. References

Building the Engine: The Symmetric "Toy" Scheme

Before we look at the complex public-key version, we start with a beautifully simple symmetric-key foundation.

Building the Engine: The Symmetric "Toy" Scheme

Before we look at the complex public-key version, we start with a beautifully simple symmetric-key foundation.

Parameters: Let η be the bit-length of the secret key, and ρ be the bit-length of the noise.

Building the Engine: The Symmetric "Toy" Scheme

Before we look at the complex public-key version, we start with a beautifully simple symmetric-key foundation.

Parameters: Let η be the bit-length of the secret key, and ρ be the bit-length of the noise.

The Symmetric Algorithms

- **KeyGen:** The secret key p is a randomly chosen odd integer $p \in [2^{\eta-1}, 2^{\eta})$.
- **Encrypt(p, m):** To encrypt a bit $m \in \{0, 1\}$, choose a large random integer q and a small random noise integer r (where $|r| < 2^{\rho}$).

$$c = p \cdot q + 2r + m$$

- **Decrypt(p, c):** Output $m' = (c \pmod{p}) \pmod 2$.

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes perfectly. We are left with $(2r + m \pmod{p})$.

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes perfectly. We are left with $(2r + m \pmod{p})$.
3. **The Critical Condition:** The modulo p operation only leaves $2r + m$ unchanged if its absolute value is strictly less than half the modulus. We strictly require:

$$|2r + m| < p/2$$

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes perfectly. We are left with $(2r + m \pmod{p})$.
3. **The Critical Condition:** The modulo p operation only leaves $2r + m$ unchanged if its absolute value is strictly less than half the modulus. We strictly require:

$$|2r + m| < p/2$$

4. Assuming the noise is small enough, we take $(2r + m) \pmod{2}$. The $2r$ is even, so it vanishes, leaving exactly m.

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes perfectly. We are left with $(2r + m \pmod{p})$.
3. **The Critical Condition:** The modulo p operation only leaves $2r + m$ unchanged if its absolute value is strictly less than half the modulus. We strictly require:

$$|2r + m| < p/2$$

4. Assuming the noise is small enough, we take $(2r + m) \pmod{2}$. The $2r$ is even, so it vanishes, leaving exactly m.

Motivation

To anyone without p, the ciphertext c just looks like a massive, random integer. But algebraically, it is a point "tethered" to a multiple of p by a short, noisy string $(2r + m)$.

Transitioning to Public-Key

A true FHE scheme requires public-key infrastructure. We achieve this by publishing a massive list of "encryptions of zero."

Transitioning to Public-Key

A true FHE scheme requires public-key infrastructure. We achieve this by publishing a massive list of "encryptions of zero."

New Parameters (based on security parameter λ):

- γ: Bit-length of the integers in the public key.
- τ: Number of integers in the public key.
- ρ': Secondary noise parameter for encryption $\rho' = \rho + \omega(\log \lambda)$.

Transitioning to Public-Key

A true FHE scheme requires public-key infrastructure. We achieve this by publishing a massive list of "encryptions of zero."

New Parameters (based on security parameter λ):

- γ: Bit-length of the integers in the public key.
- τ: Number of integers in the public key.
- ρ': Secondary noise parameter for encryption $\rho' = \rho + \omega(\log \lambda)$.

The Noise Distribution $\mathcal{D}_{\gamma,\rho}(p)$

To generate an "encryption of zero", choose $q \leftarrow \mathbb{Z} \cap [0, 2^\gamma/p)$ and $r \leftarrow \mathbb{Z} \cap (-2^\rho, 2^\rho)$. Output $x = pq + r$.

The Public-Key Algorithms

KeyGen:

- Sample $\tau + 1$ integers $x_0, x_1, \ldots, x_\tau$ from $\mathcal{D}_{\gamma,\rho}(p)$.
- Relabel so x_0 is the largest. Restart unless x_0 is odd and its noise $r_p(x_0)$ is even.
- Public key $pk = \langle x_0, x_1, \ldots, x_\tau \rangle$.

The Public-Key Algorithms

KeyGen:

- Sample $\tau + 1$ integers $x_0, x_1, \ldots, x_\tau$ from $\mathcal{D}_{\gamma,\rho}(p)$.
- Relabel so x_0 is the largest. Restart unless x_0 is odd and its noise $r_p(x_0)$ is even.
- Public key $pk = \langle x_0, x_1, \ldots, x_\tau \rangle$.

Encrypt(pk, m): Choose a random subset $S \subseteq \{1, 2, \ldots, \tau\}$ and a noise integer $r \in (-2^{\rho'}, 2^{\rho'})$.

$$c = \left[m + 2r + 2 \sum_{i \in S} x_i \right]_{x_0}$$

The Public-Key Algorithms

KeyGen:

- Sample $\tau + 1$ integers $x_0, x_1, \ldots, x_\tau$ from $\mathcal{D}_{\gamma,\rho}(p)$.
- Relabel so x_0 is the largest. Restart unless x_0 is odd and its noise $r_p(x_0)$ is even.
- Public key $pk = \langle x_0, x_1, \ldots, x_\tau \rangle$.

Encrypt(pk, m): Choose a random subset $S \subseteq \{1, 2, \ldots, \tau\}$ and a noise integer $r \in (-2^{\rho'}, 2^{\rho'})$.

$$c = \left[m + 2r + 2 \sum_{i \in S} x_i \right]_{x_0}$$

Intuition

Every x_i is effectively 0 (plus noise). Summing a random subset of x_i computes $0 + 0 + 0 \ldots$, creating a fresh, randomized encryption of zero. We multiply by 2 to keep the noise even, add our own fresh noise $2r$, and add m. The final modulo x_0 prevents the physical bit-size from growing infinitely large.

Lemma A.1: The Structure of a Fresh Ciphertext

To prove this scheme is homomorphic, we first mathematically prove that a fresh public-key ciphertext has the exact same structure as our symmetric toy scheme.

Lemma A.1: The Structure of a Fresh Ciphertext

To prove this scheme is homomorphic, we first mathematically prove that a fresh public-key ciphertext has the exact same structure as our symmetric toy scheme.

Lemma (A.1)

Let $c \leftarrow Encrypt(pk, m)$. *Then,* $c = a \cdot p + (2b + m)$ *for some integers* a *and* b *where the noise is bounded by* $|2b + m| \leq \tau 2^{\rho+2}$.

Lemma A.1: The Structure of a Fresh Ciphertext

To prove this scheme is homomorphic, we first mathematically prove that a fresh public-key ciphertext has the exact same structure as our symmetric toy scheme.

Lemma (A.1)

Let $c \leftarrow Encrypt(pk, m)$. Then, $c = a \cdot p + (2b + m)$ for some integers a and b where the noise is bounded by $|2b + m| \leq \tau 2^{\rho+2}$.

Proof Sketch:

- By definition, $c = m + 2r + 2\sum_{i \in S} x_i + k \cdot x_0$.
- We know every $x_i = q_i p + r_i$. Substitute this in:

$$c = p \cdot \left(kq_0 + 2\sum_{i \in S} q_i \right) + \left(m + 2r + 2k \cdot r_0 + 2\sum_{i \in S} r_i \right)$$

- The first term is a multiple of p (our $a \cdot p$). The second term is our accumulated noise $(2b + m)$. Its parity is m, and its magnitude is bounded.

Homomorphic Evaluation: Addition

Let the Evaluate algorithm simply be integer addition and multiplication. Let $c_1 = q_1 p + 2r_1 + m_1$ and $c_2 = q_2 p + 2r_2 + m_2$.

Homomorphic Evaluation: Addition

Let the Evaluate algorithm simply be integer addition and multiplication. Let $c_1 = q_1 p + 2r_1 + m_1$ and $c_2 = q_2 p + 2r_2 + m_2$.

Homomorphic Addition:

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (m_1 + m_2)$$

Homomorphic Evaluation: Addition

Let the Evaluate algorithm simply be integer addition and multiplication. Let $c_1 = q_1 p + 2r_1 + m_1$ and $c_2 = q_2 p + 2r_2 + m_2$.

Homomorphic Addition:

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (m_1 + m_2)$$

Observation: The structural format $a \cdot p +$ noise is perfectly preserved! We have a new multiple of p, plus $2 \times$ (new noise), plus the sum of the messages.

Crucially: The noise grows **linearly** $(r_1 + r_2)$.

Homomorphic Evaluation: Multiplication

Now, let us multiply the ciphertexts:

$$c_1 \cdot c_2 = (q_1 p + 2r_1 + m_1)(q_2 p + 2r_2 + m_2)$$

Homomorphic Evaluation: Multiplication

Now, let us multiply the ciphertexts:

$$c_1 \cdot c_2 = (q_1 p + 2r_1 + m_1)(q_2 p + 2r_2 + m_2)$$

Expanding this algebraically yields:

$$c_1 \cdot c_2 = p\big(p \cdot q_1 q_2 + q_1(2r_2 + m_2) + q_2(2r_1 + m_1)\big)$$

$$+ 2\big(2r_1 r_2 + r_1 m_2 + r_2 m_1\big) + m_1 m_2$$

Homomorphic Evaluation: Multiplication

Now, let us multiply the ciphertexts:

$$c_1 \cdot c_2 = (q_1 p + 2r_1 + m_1)(q_2 p + 2r_2 + m_2)$$

Expanding this algebraically yields:

$$c_1 \cdot c_2 = p\big(p \cdot q_1 q_2 + q_1(2r_2 + m_2) + q_2(2r_1 + m_1)\big)$$
$$+ 2\big(2r_1 r_2 + r_1 m_2 + r_2 m_1\big) + m_1 m_2$$

Observation: The structure is still preserved. We have a massive multiple of p (first term), plus $2 \times$ (mixed noise), plus the product of the messages $m_1 m_2$.

The Crisis: Look at the dominant noise term: $\mathbf{2r_1 r_2}$. When we multiply ciphertexts, the noise does not add—it **multiplies**.

The Decryption Constraint & The Noise Crisis

Lemma A.2 Constraint: Recall our strict rule for correct decryption. The modulo p operation ($c \pmod{p}$) will completely destroy the plaintext unless the absolute value of the total accumulated noise is strictly less than $p/2$.

The Decryption Constraint & The Noise Crisis

Lemma A.2 Constraint: Recall our strict rule for correct decryption. The modulo p operation ($c \pmod{p}$) will completely destroy the plaintext unless the absolute value of the total accumulated noise is strictly less than $p/2$.

The Crisis of the Square

Because multiplication squares the magnitude of the noise, the noise bit-length doubles with every sequential multiplication. It grows exponentially with the multiplicative depth of the circuit. We will rapidly hit the $p/2$ boundary.

Formalizing the Bottleneck: Lemma 3.5

Exactly how deep can our circuit be before the noise destroys the data?

Formalizing the Bottleneck: Lemma 3.5

Exactly how deep can our circuit be before the noise destroys the data?

Lemma (3.5: Permitted Polynomials)

Let $f(x_1, \ldots, x_t)$ be the multivariate polynomial computed by a circuit, and let d be its degree. The scheme can correctly evaluate this polynomial as long as:

$$d \leq \frac{\eta - 4 - \log ||\vec{f}||_1}{\rho' + 2}$$

where $||\vec{f}||_1$ is the l_1 norm of the coefficient vector of f.

Formalizing the Bottleneck: Lemma 3.5

Exactly how deep can our circuit be before the noise destroys the data?

Lemma (3.5: Permitted Polynomials)

Let $f(x_1, \ldots, x_t)$ be the multivariate polynomial computed by a circuit, and let d be its degree. The scheme can correctly evaluate this polynomial as long as:

$$d \leq \frac{\eta - 4 - \log ||\vec{f}||_1}{\rho' + 2}$$

where $||\vec{f}||_1$ is the l_1 norm of the coefficient vector of f.

Intuition: The Noise Budget

- Think of η (the bit-size of p) as our total **Noise Budget**.
- Think of ρ' (the initial noise size) as the **cost** of one multiplication.
- Therefore, the maximum multiplicative depth (degree d) we can handle is roughly our total budget divided by the cost per operation: $\approx \eta/\rho'$.

Conclusion of Phase 2

Where do we stand?

Conclusion of Phase 2

Where do we stand?

1. We have successfully built an encryption scheme that can evaluate additions and multiplications without the server knowing the underlying data.

Conclusion of Phase 2

Where do we stand?

1. We have successfully built an encryption scheme that can evaluate additions and multiplications without the server knowing the underlying data.
2. However, because of the multiplicative noise bottleneck, this scheme is strictly a **Somewhat Homomorphic Encryption (SHE)** scheme.

Conclusion of Phase 2

Where do we stand?

1. We have successfully built an encryption scheme that can evaluate additions and multiplications without the server knowing the underlying data.
2. However, because of the multiplicative noise bottleneck, this scheme is strictly a **Somewhat Homomorphic Encryption (SHE)** scheme.
3. If we evaluate a circuit deeper than $d \approx \eta/\rho'$, the noise wraps around the modulus p, and the data is lost forever.

Conclusion of Phase 2

Where do we stand?

1. We have successfully built an encryption scheme that can evaluate additions and multiplications without the server knowing the underlying data.
2. However, because of the multiplicative noise bottleneck, this scheme is strictly a **Somewhat Homomorphic Encryption (SHE)** scheme.
3. If we evaluate a circuit deeper than $d \approx \eta/\rho'$, the noise wraps around the modulus p, and the data is lost forever.

To achieve our dream of infinite FHE computation, we must find a way to evaluate our own decryption circuit within this strict noise budget. This leads us to Phase 3...

References

Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. *Fully Homomorphic Encryption over the Integers*. Eurocrypt, 2010.